



Les variables et la notion de type

B.U.T. Informatique

Objectifs d'apprentissage

- Connaître les différents types de variable
- Savoir utiliser le bon type de variable
- Savoir trouver le résultat attendu par l'opération de deux variables

La notion de variable

Une variable

Une variable est l'association d'un identificateur (c'est-à-dire un nom) à un emplacement dans la mémoire de l'ordinateur permettant de stocker une valeur

- Les variables permettent de manipuler et de stocker/représenter l'information
- Pour manipuler une variable, on utilise son nom
- Le nom ne sera pas changé

Attention

`nomVariable` $\neq$ `NOMVARIABLE`

La notion de constante

- Une constante est l'association d'un identificateur à une valeur qui ne sera **PAS** modifié
- Par convention, les constantes commencent par une majuscule

La notion de constante

- Une constante est l'association d'un identificateur à une valeur qui ne sera **PAS** modifié
- Par convention, les constantes commencent par une majuscule

Exemple

PI est une constante avec la valeur 3.14159265359

Les manipulations

Trois types de manipulation sur les variables

- Déclaration
- Affectation
- Utilisation de sa valeur

La déclaration et le typage

- Toute variable manipulée par un programme, dans un langage classique, doit avoir fait l'objet d'une déclaration
- Déclarer une variable c'est réserver un emplacement mémoire associé à son nom et à son type (typage)
- Le type d'une variable ne change pas et permet à l'ordinateur de pouvoir lire la valeur stockées
- C'est l'occasion de lui donner une valeur par défaut

Exemple

```
int x = 0;
```

L'affectation

- L'affectation c'est changer la valeur stockée d'une variable
- En C++ : `uneVariable = uneValeur`

Attention

Ne pas confondre `=` (l'affectation) et `==` (le test d'égalité).

On peut affecter une valeur en fonction de sa valeur courante ($x = x \times 3$)

Utilisation

- Si le nom d'une variable est utilisée en dehors de l'affectation ou déclaration alors c'est sa valeur que l'on récupère

Exemple

```
x = x * y + 5;
```

Le type entier

Le type entier

Il représente $\mathbb{Z}$, c'est à dire un nombre entier positif ou négatif. Les valeurs possibles sont limitées par les possibilités de l'ordinateur.

- En C++ : *int*
- Il existe aussi le type entier non signé (*unsigned int*)
- Les opérateurs de calcul : $+$, $-$, $*$, $/$, $\%$
- Les opérateurs de comparaison : $==$, $!=$, $>=$, $<=$, $>$, $<$

Exemple

```
int x = 5;  
int y = x / 2;  
int z = x * x;  
x = x+1;  
x++;  
x--;
```

x ? y ? z ?

Le type booléen

Le type booléen

Seulement deux valeurs possibles: VRAI ou FAUX. On peut appliquer des opérateurs de logique

- En C++ : *bool*
- *true* (ou 1) et *false* (ou 0)
- Les opérateurs de logique : *and*, *or*, *not*
- En C++ : *&&*, *||*, *!*

Les opérateurs logiques

x	y	x and y	x or y	not x
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Exemple

```
bool x, y, z;  
x = true;  
y = false;  
z = x && y;  
x = x || y;  
y = !y;
```

x ? y ? z ?

Le type réel

Le type réel

Il représente $\mathbf{R}$, c'est à dire un nombre réel positif ou négatif. Les valeurs possibles sont limitées par les possibilités de l'ordinateur, et la valeur est approximée.

- En C++ : *float*
- Les opérateurs de calcul : $+$, $-$, $*$, $/$
- Les opérateurs de comparaison : $==$, $!=$, $>=$, $<=$, $>$, $<$
- Opérations possibles entre réel et entier

Attention

Attention aux comparaisons entre nombres réels car l'ordinateur fait des arrondis

Exemple

```
float x = 5.0/3.0;  
int y = x;  
float z = y / x;
```

x ? y ? z ?

Le type caractère

Le type caractère

Il représente un seul caractère (lettre, chiffre, ponctuation,...). Une valeur de type caractère doit être mentionnée entre ' '

- En C++ : *char*
- Les caractères sont ordonnées : '0' < '1' < ... < 'A' < 'B' < ... < 'a' < ...
- Les opérateurs de comparaison : ==, !=, >=, <=, >, <
- La chaîne de caractère est représentée entre " "

Table ASCII

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Exemple

```
char x = '8';  
char y = x+7;
```

x ? y ?